

ANDROID STUDIO

KOTLIN- FUNTZIOEN ERREFERENTZIAK ETA LAMBDAS

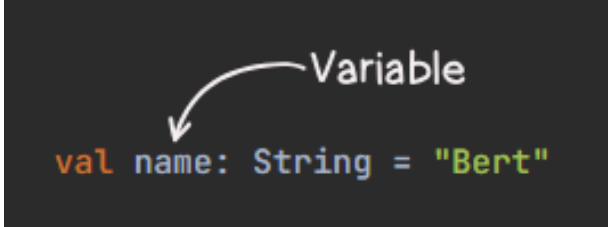
HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Funtzio motak
2. Funtzioen erreferentziak
3. Lambdak
4. It parametro inplizitua.
5. Goragoko mailako funtzioak
6. Hainbat lerrotako lambdak

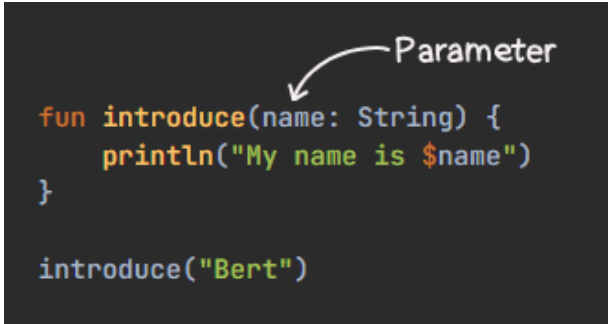
FUNTZIO MOTEN HASTAPENAK

Testua duen String aldagai sinple bat sortzeko, hau egin dezakegu:



```
val name: String = "Bert"
```

Antzera, parametroak funtzio baten aldagaiak dira, eta funtzio horretan deikodeak bere balioak pasatzen ditu funtzioa aipatzean.



```
fun introduce(name: String) {  
    println("My name is $name")  
}  
  
introduce("Bert")
```

FUNTZIO MOTEN HASTAPENAK

Orain arte, aldagai bat esleitu dugun bakoitzean edo argumentu bat funtzio batera pasatu dugun bakoitzean, String, Int eta antzeko mota sinpleetako balioekin aritu gara.

Horrelako balio sinpleak esleitzeaz gain, Kotlinek funtzioak esleitzeko aukera ere ematen dizu.

Hori frogatzeko, hemen funtzio sinple bat dago dolarretan deskontu finko bat kalkulatzeko, \$5 deskontu-kupoi bat bezala:

```
fun discountFiveDollars(price: Double): Double = price - 5.0
```

FUNTZIO MOTEN HASTAPENAK

Dakizunez, honela dei diezaiokezu funtzio honi:

```
val discountedPrice = discountFiveDollars(20.0) // Result is 15.0
```

Funtzio horri deitzeaz gain, honelako aldagai bati ere eslei diezaiokegu.

```
val applyDiscount = ::discountFiveDollars
```

FUNTZIO MOTEN HASTAPENAK

Aurreko kodean, `:: discountFiveDollars` funtzio-erreferentzia bat da. Horrela deitzen diogu funtzio bati erreferentzia egiten diolako. Funtzio hori aldagai bati esleitzean, `discountFiveDollars ()` beste izen bat ematea bezala da. Orain `applyDiscount ()` dei dezakegu, `discountFiveDollars ()` deitzen dugun bezala, eta gauza bera egiten du.

```
val discountedPrice = applyDiscount(20.0) // Result is 15.0
```

FUNTZIO MOTEN HASTAPENAK

Funtzio-erreferentzia hau idatz dezakezu ::discountFiveDollarsen, baina funtzio kide bati erreferentzia egin nahi badiozu (hau da, objektu batean dagoen funtzio bati), gehiago kalifikatu beharko duzu. Adibidez, couponForCustomer izeneko objektu bat baduzu, eta discount () izeneko funtzio bat badu, hari buruzko erreferentzia bat lor dezakezu, honela:

```
val applyDiscount = couponForCustomer::discount
```

FUNTZIO MOTEN HASTAPENAK

Gogoratuko duzuenez, aldagai bat deklaratzeko dugun bakoitzean, aldagai horrek mota bat du, baita kodean esplizituki idatzita ez badago ere. Adibidez:

```
val name = "Bert"    // name's type is String
val hasCoupon = true // hasCoupon's type is Boolean
val price = 12.50    // price's type is Double
```

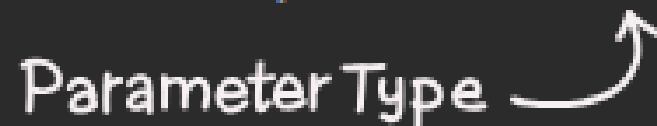

FUNTZIO MOTEN HASTAPENAK

Orduan, agian galdetuko diozue zeure buruari zein den applyDiscountvariable mota.

Aldagai batek funtzio bat duenean, bere parametro mota guztien eta itzulera-motaren konbinazioa da. `DiscountFiveDollars()` funtzioaren kasuan, hauek Double motako parametro bakarra eta Double motako itzulera-mota bat dituzte.

```
fun discountFiveDollars(price: Double): Double = price - 5.0
```

Parameter Type



Return Type



FUNTZIO MOTEN HASTAPENAK

Funtzio mota bat honela mihizta(ensamblar) daiteke:

1. Parentesiak mantenduz
2. Motak mantenduz
3. Bi puntuak gezi bihurtzea ->.

Egin dezagun `discountFiveDollarsekin ()`.

```
fun discountFiveDollars(price: Double): Double = price - 5.0
```

(Double) -> Double

FUNTZIO MOTEN HASTAPENAK

Beraz, `discountFiveDollars()`-en mota (`Double`) -> `Double`. Hori jakinda, orain modu esplizituan idatz dezakezu `applyDiscount` mota:

```
val applyDiscount: (Double) -> Double = ::discountDollars
```

Orain arte ikusi ditugun tipo guztiek ez zuten espaziorik, `String`, `Int` eta `Double`. Ikus dezakezunez, `(Double) -> Double` bezalako funtzio motak luze samarrak dira, baina ulerterrazak! Parametro motak parentesien barruan daude, eta emaitza geziaren eskuinaldean dago.

Parameter type(s)
inside parentheses

Result type to the
right of the arrow

`(Double) -> Double`

FUNTZIO MOTEN HASTAPENAK

Hainbat parametro dituzten funtzioen kasuan, parametro motak koma batekin bitartez bereiziko dira. Adibidez, jarraian bi parametro dituen funtzio bat agertzen da (String eta Double) eta String bat itzultzen du. Funtzio hori aldagai bati esleitzen zaionean, aldagai mota hau izango da: **(String, Double) -> String**

```
fun menuItemDescription(name: String, price: Double): String =  
    "A $name costs $price before discounts and tax."  
  
val describeMenuItem: (String, Double) -> String = ::menuItemDescription
```

FUNTZIO MOTEN HASTAPENAK

Mota bereko bi funtzio

Gogoratuko duzuenez, mota bereko bi balio badituzue, hala nola bi String, bi balore horiek aldagai berari esleitu (eta berresleitu) ahal dizkiozue.

```
var couponCode = "FIVE_BUCKS"  
couponCode = "TAKE_10"
```

Antzera gertatzen da mota bereko bi funtzio baditugu (hau da, parametro mota eta itzulera mota berberak dituzten funtzioak), bi funtzio horiek aldagai berari esleitu (eta berresleitu) ahal dizkio.

FUNTZIO MOTEN HASTAPENAK

Hori frogatzeko, sar dezagun %10eko deskontu-kupoi baten deskontua kalkulatzeko funtzio bat. DiscountFiveDollars()-en parametro eta emaitza mota berak dituenenez, funtzio horietako edozein aldagai berari esleitu ahal diogu.

```
fun discountFiveDollars(price: Double): Double = price - 5.0
fun discountTenPercent(price: Double): Double = price * 0.9

var applyDiscount = ::discountFiveDollars
applyDiscount = ::discountTenPercent
```

FUNTZIO MOTEN HASTAPENAK

Hori frogatzeko, sar dezagun %10eko deskontu-kupoi baten deskontua kalkulatzeko funtzio bat. DiscountFiveDollars()-en parametro eta emaitza mota berak dituenenez, funtzio horietako edozein aldagai berari esleitu ahal diogu.

```
fun discountFiveDollars(price: Double): Double = price - 5.0
fun discountTenPercent(price: Double): Double = price * 0.9

var applyDiscount = ::discountFiveDollars
applyDiscount = ::discountTenPercent
```

Parametro asko dituzten funtzioen kasuan, kontuan izan parametroek ordena berean bat etorri behar dutela.

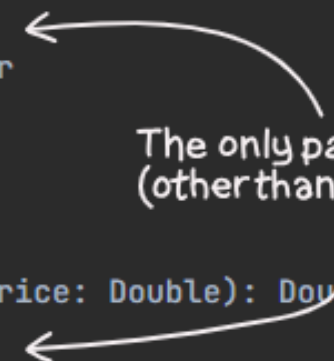
FUNTZIOEN ERREFERENTZIAK

Funtzioak funtzioetara pasatzea

Kontutan izango dugu ondorengo adibidea. Lehenngoan 5\$-eko deskontua izango du bigarreanean %10. Hori da bien arteko diferentzia bakarra.

```
fun calculateTotalWithFiveDollarDiscount(initialPrice: Double): Double {  
    val priceAfterDiscount = initialPrice - 5.0  
    val total = priceAfterDiscount * taxMultiplier  
  
    return total  
}  
  
fun calculateTotalWithTenPercentDiscount(initialPrice: Double): Double {  
    val priceAfterDiscount = initialPrice * 0.9  
    val total = priceAfterDiscount * taxMultiplier  
  
    return total  
}
```

The only part that differs
(other than the name)



FUNTZIOEN ERREFERENTZIAK

Argudio gisa kodea pasatzeko modu bakarra aurkituko bagenu... orduan gure bi funtzioak funtzio bakar batean finkatu ahal izango genituzke. Funtzio hori honela ikusiko litzateke:

```
fun calculateTotal(  
    initialPrice: Double,  
    applyDiscount: ???  
): Double {  
    val priceAfterDiscount = applyDiscount(initialPrice)  
    val total = priceAfterDiscount * taxMultiplier  
  
    return total  
}
```

FUNTZIOEN ERREFERENTZIAK

Orain funtzio motak zer diren dakigunez, calculateTotal() funtzioa eguneratu egingo dugu, aldaketa txiki batzuk egingo dizkiogu (Double) -> Double funtzioa duen applyDiscount izeneko parametro bat onar dezan.

```
fun calculateTotal(  
    initialPrice: Double,  
    applyDiscount: (Double) -> Double  
): Double {  
    // Apply coupon discount  
    val priceAfterDiscount = applyDiscount(initialPrice)  
    // Apply tax  
    val total = priceAfterDiscount * taxMultiplier  
  
    return total  
}
```

FUNTZIOEN ERREFERENTZIAK

Kode horren ordeaz, dagoeneko kalkula daiteke calculateTotal() erreferentzia_function batekin. Defini ditzagun beste funtzio batzuk (Double) -> Double motarekin bat datozenak eta, ondoren, kalkula dezagun calculateTotal() horietako bakoitzarekin:

```
fun discountFiveDollars(price: Double): Double = price - 5.0
fun discountTenPercent(price: Double): Double = price * 0.9
fun noDiscount(price: Double): Double = price

val withFiveDollarsOff = calculateTotal(20.0, ::discountFiveDollars) // $16.35
val withTenPercentOff  = calculateTotal(20.0, ::discountTenPercent) // $19.62
val fullPrice          = calculateTotal(20.0, ::noDiscount)           // $21.80
```

FUNTZIOEN ERREFERENTZIAK

Funtzioak funtzioetatik itzultzea

Funtzioaren izena idatzi beharrean, calculateTotal() deitzen duen bakoitzean, zelan izango litzateke bezeroarengandik jasotzen duen kupoiaren beheko aldean agertzen den kupoi-kodea sartzen badugu?



FUNTZIOEN ERREFERENTZIAK

Horretarako, kupoi-kodea onartzen duen eta deskontu-funtzio egokia itzultzen duen funtzio bakarra behar du. Bestela esanda, String bat izango den parametro bat izango du, eta itzulera-mota (Double) -> Double izango da.

When hitz erreserbatu batek funtzio hau erraz bihurtzen du!

```
fun discountForCouponCode(couponCode: String): (Double) -> Double = when (couponCode) {  
    "FIVE_BUCKS" -> ::discountFiveDollars  
    "TAKE_10"    -> ::discountTenPercent  
    else         -> ::noDiscount  
}
```

FUNTZIOEN ERREFERENTZIAK

Eguneraketa:

```
fun discountFiveDollars(price: Double): Double = price - 5.0
fun discountTenPercent(price: Double): Double = price * 0.9
fun noDiscount(price: Double): Double = price

val withFiveDollarsOff = calculateTotal(20.0, ::discountFiveDollars) // $16.35
val withTenPercentOff  = calculateTotal(20.0, ::discountTenPercent) // $19.62
val fullPrice          = calculateTotal(20.0, ::noDiscount)          // $21.80
```

```
val withFiveDollarsOff = calculateTotal(20.0, discountForCouponCode("FIVE_BUCKS")) // $16.35
val withTenPercentOff  = calculateTotal(20.0, discountForCouponCode("TAKE_10"))   // $19.62
val fullPrice          = calculateTotal(20.0, discountForCouponCode("NONE"))      // $21.80
```

FUNTZIOEN ERREFERENTZIAK

Aurreko `calculateTotal()` eta `discountForCouponCode ()` bezalako funtzioei, argumentu gisa funtzioak onartzen dituztenei eta/edo emaitza gisa itzultzen dituztenei, **goragoko mailako funtzioak** esaten zaie.

This is a higher-order function because it accepts a function.



```
fun calculateTotal(initialPrice: Double, applyDiscount: (Double) -> Double): Double {  
    // Apply coupon discount  
    val priceAfterDiscount = applyDiscount(initialPrice)  
    // Apply tax  
    val total = priceAfterDiscount * taxMultiplier  
  
    return total  
}
```

FUNTZIOEN ERREFERENTZIAK

This is a higher-order function because it returns a function.



```
fun discountForCouponCode(couponCode: String): (Double) -> Double = when (couponCode) {  
    "FIVE_BUCKS" -> ::discountFiveDollars  
    "TAKE_TEN"   -> ::discountTenPercent  
    else         -> ::noDiscount  
}
```

Orain arte, asko lortu ahal izan dugu funtzioen erreferentziekin. Nahiko erabilgarriak izan daitezke aipatu nahi duzun funtzioa idatzi duzunean. Baina Kotlinetik aldagaiei eta parametroei funtzioak esleitzeko beste modu zehatzago bat ere ematen digu: **lambdak**!

LAMBDETARAKO SARRERA

Gogoratuko duzuenez, literala da zure kodean balio bat zuzenean idazten duzuenean. Adibidez, Kotlinen, String, Int eta Boolean bezalako oinarritzko motetarako literalak sor ditzakezue. Kodearen zati nabarmenduak, jarraian, literal gisa idazten diren balioak dira.

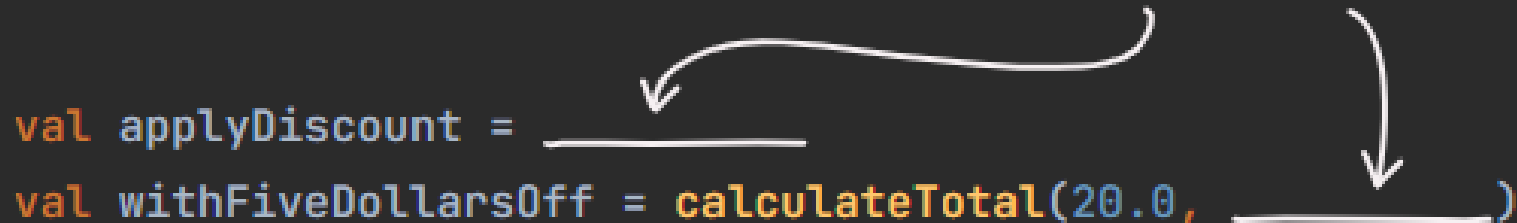
```
val string: String = "This is a string"  
val integer: Int = 49  
val boolean: Boolean = true
```

LAMBDETARAKO SARREERA

Kate-literalak, osoak eta boolearrak idatz ditzakezuen bezala, funtzio baten literala ere idatz dezakezue.

Orainarte fun gako-hitzarekin funtzioak idazten aritu gara, baina inoiz ez dugu funtzio bat zuzenean definitu adierazpen batean, esleipen baten eskuinaldean bezala edo zuzenean funtzio-dei baten barruan.

We want to define a function directly here or here.



```
val applyDiscount = _____  
val withFiveDollarsOff = calculateTotal(20.0, _____)
```

The diagram shows two lines of code. The first line is `val applyDiscount = _____`. The second line is `val withFiveDollarsOff = calculateTotal(20.0, _____)`. A curved arrow points from the text "We want to define a function directly here or here." to the underscore in the first line. Another arrow points from the same text to the underscore in the second line.

LAMBDETARAKO SARRERA

Lehengo adibidearekin jarraituz:

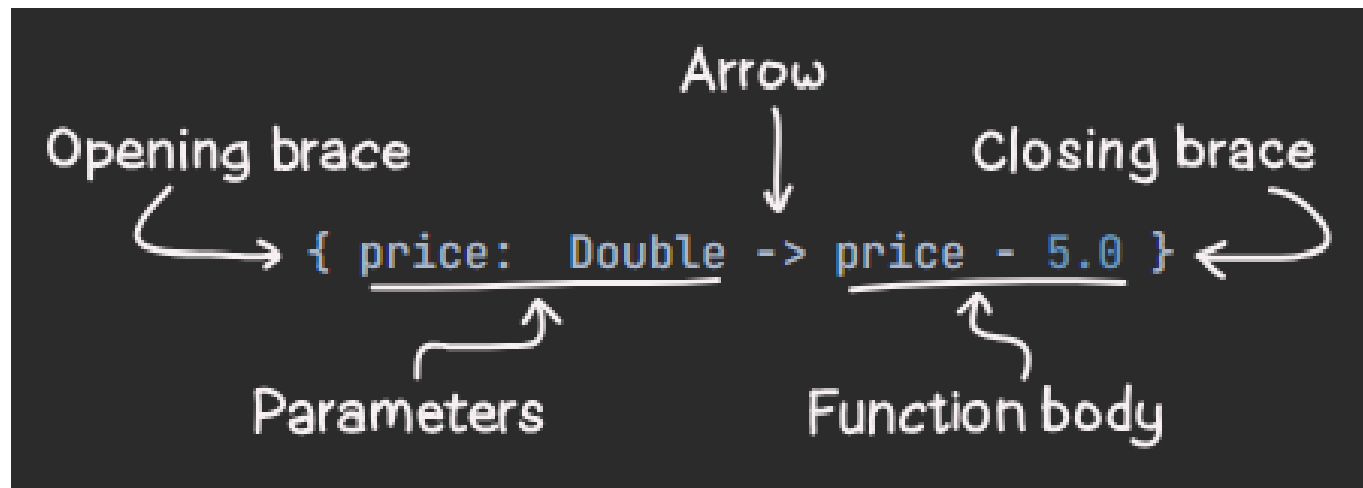
```
fun discountFiveDollars(price: Double) = price - 5.0  
val applyDiscount: (Double) -> Double = ::discountFiveDollars
```

discountFiveDollars() funtzioa fun gako-hitzarekin definitu beharrian, funtzio-literal gisa berridatz dezakegu, honela:

```
val applyDiscount: (Double) -> Double = { price: Double -> price - 5.0 }
```

LAMBDETARAKO SARRERA

Aurreko kodean nabarmendutako zatia **funtzioaren literala** da. Kotlin-en, modu horretan idatzitako funtzio-literal bati **lambda** deitzen zaio. Lambda-ak orain arte idatzi ditugunak bezalako funtzioak dira baina modu desberdinean adierazten dira. Lambda bat idazteko, erabili irekitzeko giltza {eta ixteko giltza}. Gero, idatzi parametroak geziaren aurretik -> eta gorputza geziaren ondoren.



LAMBDETARAKO SARRERA

```
val applyDiscount: (Double) -> Double = { price -> price - 5.0 }
```

Kotlinek badaki Price-k Double bat izan behar duela, hori baita applyDiscount-ek izan behar duen adierazlea. Era berean, lambdaren emaitzak kointzididu behar du.

```
val applyDiscount: (Double) -> Double = { price -> price - 5.0 }
```

Parameter Type

Result Type

IT PARAMETRO INPLIZITUA

Lambda baterako parametro bakarra dagoen kasuetan, parametroaren izena eta gezia alde batera utz ditzakezue. Hau egiten duzuenean, Kotlinek automatikoki sortuko du it parametroaren izena. Berridatz dezagun gure lambda hau aprobeatzeko:

```
val applyDiscount: (Double) -> Double = { it - 5.0 }
```

LAMBDAK ETA GORAGOKO MAILAKO FUNTZIOAK

Lambdak argumentu gisa igaroz

Lehen ikasi dugunez, goi-ordenako funtzioak sarrera (hau da, parametro) edo irteera (hau da, emaitza) funtzioa dutenak dira.

Hona hemen aurreko adibideen kodea, non funtzio-erreferentziak erabiltzen ditugun ,funtzioak, argumentu gisa calculateTotal() funtzioan.

LAMBDAK ETA GORAGOKO MAILAKO FUNTZIOAK

```
fun calculateTotal(
    initialPrice: Double,
    applyDiscount: (Double) -> Double
): Double {
    // Apply coupon discount
    val priceAfterDiscount = applyDiscount(initialPrice)
    // Apply tax
    val total = priceAfterDiscount * taxMultiplier

    return total
}

fun discountFiveDollars(price: Double) = price - 5.0
fun discountTenPercent(price: Double): Double = price * 0.9
fun noDiscount(price: Double) = price

val withFiveDollarsOff = calculateTotal(20.0, ::discountFiveDollars) // $16.35
val withTenPercentOff  = calculateTotal(20.0, ::discountTenPercent)  // $19.62
val fullPrice          = calculateTotal(20.0, ::noDiscount)          // $21.80
```


LAMBDAK ETA GORAGOKO MAILAKO FUNTZIOAK

Erraza da `calculateTotal()` deitzea lambda batekin, funtzio-erreferentzia baten ordeztuz. Berridatz ditzagun aurreko kodearen azken lerroak lambdak erabiltzeko. Funtzio bakoitzaren gorputza hartu eta lambda bat bezala idatziko dugu:

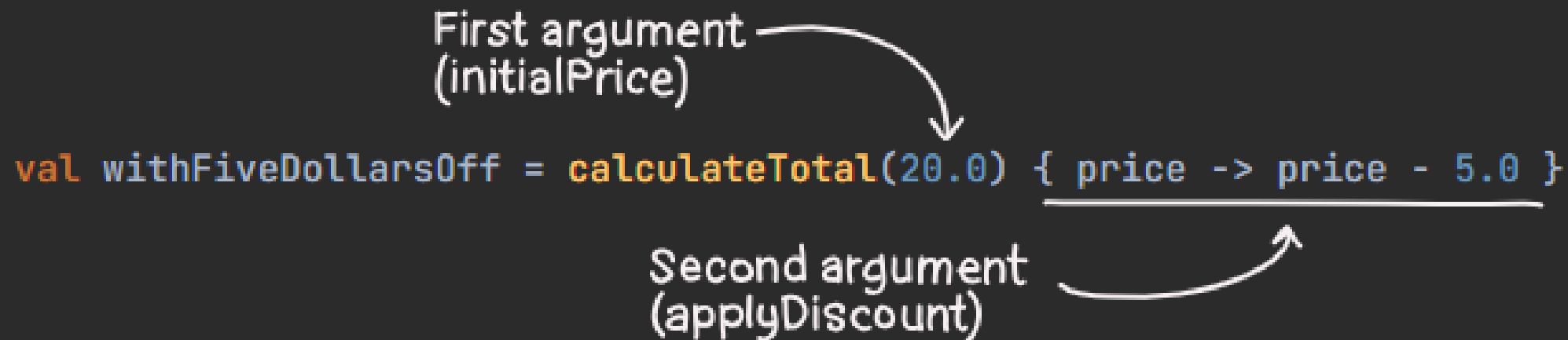
```
val withFiveDollarsOff = calculateTotal(20.0, { price -> price - 5.0 }) // $16.35
val withTenPercentOff  = calculateTotal(20.0, { price -> price * 0.9 }) // $19.62
val fullPrice          = calculateTotal(20.0, { price })                // $21.80
```

Funtzio baten azken parametroa funtzio mota bat denean, parentesietatik kanpo lambda argumentua eskuinerantz mugi dezakezu, honela:

```
val withFiveDollarsOff = calculateTotal(20.0) { price -> price - 5.0 }
val withTenPercentOff  = calculateTotal(20.0) { price -> price * 0.9 }
val fullPrice          = calculateTotal(20.0) { price -> price }
```

LAMBDAK ETA GORAGOKO MAILAKO FUNTZIOAK

Bi argudio bidaltzen jarraitzen dugu `calculateTotal()` Lehenengoa parentesien barruan dago eta bigarrena kanpoan, eskuinean.



```
val withFiveDollarsOff = calculateTotal(20.0) { price -> price - 5.0 }
```

The diagram illustrates the arguments of the `calculateTotal` function. The first argument, `20.0`, is labeled "First argument (initialPrice)" with an arrow pointing to it. The second argument, the lambda function `{ price -> price - 5.0 }`, is labeled "Second argument (applyDiscount)" with an arrow pointing to it.

LAMBDAK ETA GORAGOKO MAILAKO FUNTZIOAK

Parametro bakarra duen goi-ordenako funtzio bat dago, eta funtzio mota hau du:

```
fun printSubtotal(applyDiscount: (Double) -> Double) {  
    val result = applyDiscount(20.0)  
    val formatted = "%.2f".format(result)  
    println("A $20.00 haircut will cost you $formatted before tax.")  
}
```

PrintSubtotal () deitzean, ez da parentesirik behar!

```
printSubtotal { price -> price - 5.0 }  
printSubtotal { price -> price * 0.9 }
```

LAMBDAK ETA GORAGOKO MAILAKO FUNTZIOAK

Lambdak itzultzea funtzioaren emaitza gisa

Argudio gisa lambdak erabiltzeaz gain, funtzioen emaitza gisa ere erabil ditzakegu!

Ondorengo kodigoan oinarrituz:

```
fun discountForCouponCode(couponCode: String): (Double) -> Double = when (couponCode) {  
    "FIVE_BUCKS" -> ::discountFiveDollars  
    "TAKE_10"    -> ::discountTenPercent  
    else         -> ::noDiscount  
}
```

LAMBDAK ETA GORAGOKO MAILAKO FUNTZIOAK

Erraz ordezkia ditzakegu funtzio-erreferentzia horiek lambdekin

```
fun discountForCouponCode(couponCode: String): (Double) -> Double = when (couponCode) {  
    "FIVE_BUCKS" -> { price -> price - 5.0 }  
    "TAKE_10"    -> { price -> price * 0.9 }  
    else         -> { price -> price }  
}
```

HAINBAT LERROTAKO LAMBDAK

Orain arte, erabili ditugun lambdek adierazpen single bana baino ez zuten. Batzuetan, lambda bat behar duzu, sententzia batzuk dituen. Horretarako, sententzia bakoitza lerro bereizi batean jartzen du, beste edozein funtzioaren barruan egingo zenukeen bezala. Hala ere, funtzio normal batean ez bezala, ez duzu return gako-hitza erabiliko emaitza itzultzeko. Lambdaren azken lerroa, berriz, deiaren emaitza izango da.

Adibidez, baliteke arazketa-informazioaren (depuración) bat inprimatu nahi izatea bost dolarreko deskontu-kupoia kalkulatzeko duen lambda barruan:

HAINBAT LERROTA KO LAMBDAK

```
val withFiveDollarsOff = calculateTotal(20.0) { price ->
    val result = price - 5.0
    println("Initial price: $price")
    println("Discounted price: $result")
    result
}
```

Horrelako lerro bat baino gehiago hartzen dituen lambda bat dugunean, parametroak eta gezia irekiera-giltzaren lerro berean jarri ohi dira, goian ikusten den bezala. Hemen dago kode bera, zati bakoitza adierazten duten ohar batzuekin.

ITXIERA

Orainarte ikusitakoarekin, bezeroen guztizkoak erraz kalkula ditzakegu, baita kupoi desberdinak dituztenean ere. Begiratu zure kodea, barne `calculateTotal()`, `discountForCouponCode ()`, eta nola azkenean deitu guztizkoa lortzeko.

ITXIERA

```
fun calculateTotal(
    initialPrice: Double,
    applyDiscount: (Double) -> Double
): Double {
    // Apply coupon discount
    val priceAfterDiscount = applyDiscount(initialPrice)
    // Apply tax
    val total = priceAfterDiscount * taxMultiplier

    return total
}

fun discountForCouponCode(couponCode: String): (Double) -> Double = when (couponCode) {
    "FIVE_BUCKS" -> { price -> price - 5.0 }
    "TAKE_10"    -> { price -> price * 0.9 }
    else         -> { price -> price }
}

val initialPrice = 20.0
val couponDiscount = discountForCouponCode("FIVE_BUCKS")
val total = calculateTotal(initialPrice, couponDiscount)
```

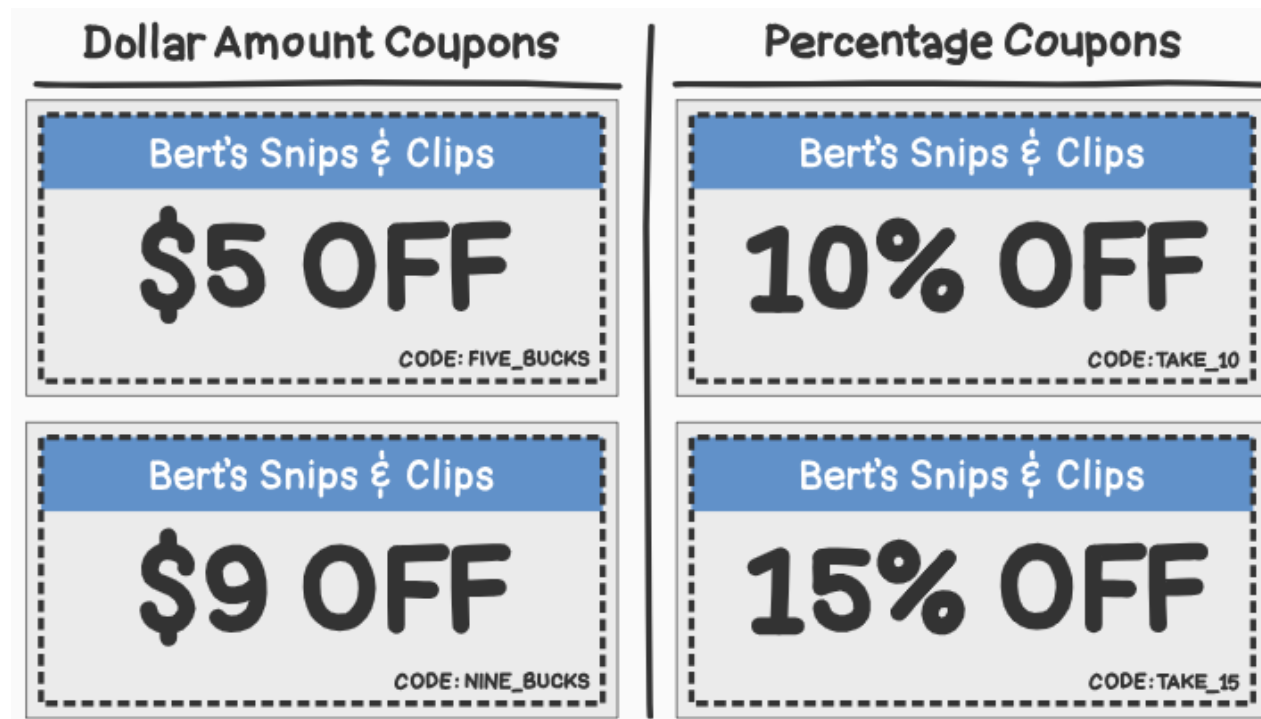
ITXIERA

Kodea begiratura konturatsen gara kupoi berri bat sartzen dugunean beste lambda bat idatzi behar dugula. Adibidez, bederatzi dolarreko deskontua duen kupoi berri bat eta ehuneko hamabosteko deskontua duen beste bat gehituz gero, beste lambda batzuk idatzi beharko dituzu, hala nola:

```
fun discountForCouponCode(couponCode: String): (Double) -> Double = when (couponCode) {  
    "FIVE_BUCKS" -> { price -> price - 5.0 }  
    "NINE_BUCKS" -> { price -> price - 9.0 }  
    "TAKE_10"    -> { price -> price * 0.9 }  
    "TAKE_15"    -> { price -> price * 0.85 }  
    else         -> { price -> price }  
}
```

ITXIERA

Egia esan, ez dago hain gaizki, baina azken hobekuntza txiki bat egin genezake. Egia esan, kupoiak bi kategoria nagusi daude: zenbatekoa dolarretan eta ehunekoa.



ITXIERA

Bi funtzio hauek idatziko ditugu, identifikatu ditugun kupoien bi kategoriekin bat etor daitezen:

```
fun dollarAmountDiscount(dollarsOff: Double): (Double) -> Double =  
    { price -> price - dollarsOff }  
  
fun percentageDiscount(percentageOff: Double): (Double) -> Double {  
    val multiplier = 1.0 - percentageOff  
    return { price -> price * multiplier }  
}
```

ITXIERA

Garrantzitsua da kontuan izatea bi funtzio horiek ez dutela deskontua beren kabuz kalkulatzeko, baizik eta deskontua kalkulatzeko funtzioak sortzen dituztela. Hau pixka bat errazagoa da `percentageDiscount()`, non `return` gako-hitz esplizitu bat erabiltzen dugu adierazpen-gorputz baten ordeiz.

Hemen beste gauza interesgarri bat da lambda hauek lambda gorputzetik kanpo definituta dauden aldagaiak erabiltzen dituztela. Lehenengoak `dollarsOffvariable` erabiltzen du (funtzio inguratzailearen parametro bat) eta bigarrenak `multiplier` aldagaia. Lambda batek bere gorputzetik kanpo hau bezala definituta dagoen aldagai bat erabiltzen duenean, batzuetan **itxiera** esaten zaio.

ITXIERA

Orain, kupoi berri bat sortzea erraza da. Lambda bat idatzi ordez online `discountForCouponCode()`-an, besterik gabe `dollarAmountDiscount ()` edo `percentageDiscount ()` deitu ahal izango zaio lambda sortzeko.

```
fun discountForCouponCode(couponCode: String): (Double) -> Double = when (couponCode) {  
    "FIVE_BUCKS" -> dollarAmountDiscount(5.0)  
    "NINE_BUCKS" -> dollarAmountDiscount(9.0)  
    "TAKE_10"    -> percentageDiscount(0.10)  
    "TAKE_15"    -> percentageDiscount(0.15)  
    else         -> { price -> price }  
}
```